

Instructions

1 Installation

1.1 Installing GAP

Open a terminal in Ubuntu and navigate to the directory where you want to install GAP. Then run the following commands.

1. Install dependencies

```
sudo apt update
sudo apt install build-essential gcc g++ make m4 zlib1g-dev libgmp-dev wget
```

2. Download and extract GAP

```
wget https://github.com/gap-system/gap/releases/download/v4.14.0/gap-4.14.0.tar.gz
tar -xzf gap-4.14.0.tar.gz
cd gap-4.14.0
```

Inside the gap folder copy and past the gap file given in the github repo.

1.2 Copying the Code files

Inside your folder where gap is present , create a new folder, you can name it anything say "Codes" and copy all the .g and .py files given in the github folder Codes.

1.3 Installing Python

1. Install Python and virtual environment support Open a separate terminal and follow the below steps.

```
sudo apt install python3
sudo apt install python3-venv
```

2. Create and activate a virtual environment

```
python3 -m venv venv
source venv/bin/activate
```

3. Install required packages

```
pip install numpy sympy
```

you can close the terminal after the installation.

2 Loading the Algebra

Once everything is installed, we are ready to dive into the computation. We start by loading the algebra, for which you need to create two files: a `.g` file and a `.py` file. Give them the same name to avoid confusion, or simply load your algebra into one of the existing file pairs, e.g., `Algebra.g` with its corresponding `allrels.py`, or `A-ZAlgebras.g` with its corresponding `A-ZAlgebras.py`. There are approximately 68–73 preloaded algebras along with their relations. The format for loading algebras is self-explanatory and can be inferred directly from the existing code. Note the variable type in `.g` and `typ` in python should have the same name.

Once the algebra is loaded (with relations) we are ready to run the code.

2.1 Main.g

Open the `Main.g` file. Here we call the function that creates the quiver. In the code, you will see:

```
Read("Codes/ALGEBRAS.g");
```

This is where the quotient of the path algebra gets loaded. Provide the correct path name according to your directory structure:

```
main_path := "/gap-4.14.0/Codes/";
```

This is the path inside the GAP folder where your code files reside, and it is also where the results will be stored.

```
mod_spec := ComputeModuleSpec( A, type, main_path );
```

This function computes the action of the module, allowing you to construct a module over the path algebra. Here, `A` is the algebra loaded from `Algebras.g`, and `type` is also specified in the `.g` file. You will find a folder named `"type"`, containing a `mod_spec` file inside it.

In principle, the whole process could be carried out using the single built-in function `AlgebraAsModuleOverEnvelopingAlgebra(A)`. However, this function only works when the algebra is simple, in the sense that it has no relations. Since our algebras involve substantial relations, this built-in function does not directly apply. We therefore examined the source code of `AlgebraAsModuleOverEnvelopingAlgebra` and modified it to work correctly for algebras with relations.

The output of `mod_spec` is not directly in the form we want for the next stage of the computation. We require the data in the form of vectors, which is achieved using Python. This modified function serves the role of the built-in `Coefficients` function, but for algebras with relations. A sample output for an algebra can be found on GitHub.

2.2 Python inside gap

```
Exec("/myenv/bin/python3 /gap-4.14.0/Codes/Coefficients.py");
```